



Pashov Audit Group

Hypernova Security Review

March 9th 2026 - March 10th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification.....	3
4. About Hypernova	4
5. Executive Summary	4
6. Findings	5
Low findings	6
[L-01] MaxWithdrawalLimit is enforced on pre-split payout amount.....	6
[L-02] <code>getMaxWithdrawable</code> does not account for withdrawal constraints.....	6
[L-03] Accept and reject functions do not emit status event.....	7
[L-04] Payout can be overwritten by later admin equity update	7
[L-05] Stale withdrawal authorization after equity update	8
[L-06] <code>setUserBonusSplit</code> has no upper bound, may block payouts.....	8



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Hypernova

Hypernova is a smart contract system for managing funded trading accounts with on-chain equity tracking, settlement, and payout processing. It uses a USDC Vault to handle deposits, withdrawals, and configurable profit splitting between traders and the platform.

5. Executive Summary

A time-boxed security review of the **hypernova-core/smart-contracts** repository was done by Pashov Audit Group, during which **Blockace**, **JCN**, **t.aksoy** engaged to review **Hypernova**. A total of **6** issues were uncovered.

Protocol Summary

Project Name	Hypernova
Protocol Type	Funded trading accounts
Timeline	March 9th 2026 - March 10th 2026

Review commit hash:

- [88c0d504b9388410fd7966f0c6acd452a975b39d](#)
(hypernova-core/smart-contracts)

Fixes review commit hash:

- [eefd50f4eb62dc0cb4fe8ac2fcee16784ec7dfd9](#)
(hypernova-core/smart-contracts)

Scope

`TradingAccounts.sol``Vault.sol`



6. Findings

Findings count

Severity	Amount
Low	6
Total findings	6

Summary of findings

ID	Title	Severity	Status
[L-01]	MaxWithdrawalLimit is enforced on pre-split payout amount	Low	Resolved
[L-02]	<code>getMaxWithdrawable</code> does not account for withdrawal constraints	Low	Resolved
[L-03]	Accept and reject functions do not emit status event	Low	Resolved
[L-04]	Payout can be overwritten by later admin equity update	Low	Resolved
[L-05]	Stale withdrawal authorization after equity update	Low	Resolved
[L-06]	<code>setUserBonusSplit</code> has no upper bound, may block payouts	Low	Resolved



Low findings

[L-01] MaxWithdrawalLimit is enforced on pre-split payout amount

Description

maxWithdrawalLimit is described as the maximum amount that can be withdrawn.

```
/// @notice Maximum amount that can be withdrawn in a single transaction
uint256 public maxWithdrawalLimit;
```

In processPayout, the limit check uses `(if (_amount > maxWithdrawalLimit))` before the split, while the user actually receives traderAmount after the split. This makes the configured limit stricter than it appears for traders.

Apply maxWithdrawalLimit to trader Amount

[L-02] getMaxWithdrawable does not account for withdrawal constraints

Description

`getMaxWithdrawable` returns raw profit without reflecting the constraints that `requestPayout` enforces:

```
function getMaxWithdrawable(address _trader, uint256 _tradingAccountIndex) external view
returns (uint256) {
    if (tradingAccountCounts[_trader] == 0) revert UserAccountDoesNotExist();
    if (_tradingAccountIndex >= tradingAccountCounts[_trader]) revert
InvalidTradingAccountIndex();

    TradingAccount storage sub = tradingAccounts[_trader][_tradingAccountIndex];

    if (sub.equity <= sub.initialEquity) {
        return 0;
    }

    return sub.equity - sub.initialEquity;
}
```

The above function ignores the `canWithdraw` flag, `fundedStatus`, user suspension, the vault's `maxWithdrawalLimit`, the vault's pause state, and the vault's USDC balance. Integrators or front ends relying on this function may display amounts the trader cannot actually withdraw.

Consider documenting these limitations for integrators, or incorporating the additional constraints so the return value reflects the actual withdrawable amount.



Recommendations

At minimum, cap the return value at `maxWithdrawalLimit` so the displayed amount never exceeds what `processPayout` will permit: `return profit > maxWithdrawalLimit ? maxWithdrawalLimit : profit;` where `profit = sub.equity - sub.initialEquity`.

[L-03] Accept and reject functions do not emit status event

Description

When the admin changes the funded status via `updateFundedStatus`, a `FundedStatusUpdated` event is emitted. On the other hand, when a trader accepts or rejects a funded account, specific events are emitted that do not directly emit the new status update:

```
function acceptFundedAccount(uint256 _tradingAccountIndex)
    external
    whenNotSuspended(msg.sender)
    onlyAwaitingSignature(_tradingAccountIndex)
{
    tradingAccounts[msg.sender][_tradingAccountIndex].fundedStatus = FundedStatus.ACTIVE;
    emit FundedAccountAccepted(msg.sender, _tradingAccountIndex);
}

function rejectFundedAccount(uint256 _tradingAccountIndex)
    external
    whenNotSuspended(msg.sender)
    onlyAwaitingSignature(_tradingAccountIndex)
{
    tradingAccounts[msg.sender][_tradingAccountIndex].fundedStatus = FundedStatus.CLOSED;
    emit FundedAccountRejected(msg.sender, _tradingAccountIndex);
}
```

An off-chain indexer tracking funded status updates via `FundedStatusUpdated` would miss trader-initiated status transitions entirely. The status would appear stuck at `AWAITING_SIGNATURE` even after the on-chain state has moved to `ACTIVE` or `CLOSED`. In order to accurately track the funded status of an account, indexers would need to track three separate event types.

Consider also emitting `FundedStatusUpdated` in both functions so all funded status transitions are trackable through a single event type.

[L-04] Payout can be overwritten by later admin equity update

Description

`requestPayout` changes equity relatively (`equity -= amount`), while admin update functions can later set equity absolutely (`equity = newValue`).



If the user frontruns the equity update with `requestPayout`, a stale admin overwrite can remove the effect of a payout debit.

Attack Scenario

1. Admin settles equity at `12,000` (initial `10,000`) and sets `canWithdraw = true`.
2. User does not withdraw yet and watches the mempool for admin `updateEquity`.
3. Admin later sends `updateEquity` with value `13,000`.
4. User front-runs `requestPayout(1000)` while the window is open.
5. `requestPayout` passes `profit` check against `12,000` and writes storage `equity = 11,000`.
6. Admin overwrites lands with `equity = 13,000`, so payout was allowed on a stale state.

Recommendations

Instead of absolute value update, use increase/decrease.

[L-05] Stale withdrawal authorization after equity update

Description

`updateEquityAndSettle` sets `canWithdraw = true`, but later equity changes do not reset it.

After settlement, the admin can call `updateEquity`, `batchUpdateEquity`, etc., while `canWithdraw` may remain true.

Then `requestPayout` uses the new equity values with the old authorization flag.

This can allow payout from an equity state that was not explicitly settled for withdrawal.

Recommendations

Clear `canWithdraw` on every non-settlement equity mutation.

[L-06] `setUserBonusSplit` has no upper bound, may block payouts

Description

The `setUserBonusSplit` function updates the `userBonusBps` mapping for users without any bounds validation:

```
function setUserBonusSplit(address _trader, uint256 _bonusBps) external onlyAdmin
whenNotSuspended(_trader) {
    if (tradingAccountCounts[_trader] == 0) revert UserAccountDoesNotExist();
```



```
userBonusBps[_trader] = _bonusBps;  
emit UserBonusSplitUpdated(_trader, _bonusBps);  
}
```

The `Vault::processPayout` function will revert if `profitSplit + _bonusBps > BPS_DENOMINATOR`. This means the admin can set a bonus that causes a trader's payouts to revert. A settled trader with `canWithdraw = true` would be unable to withdraw.

It is important to note that the check in `processPayout` can also revert after a valid bonus is set if the vault owner then increases the `profitSplit` such that a trader's `bonus + new_profitSplit > BPS_DENOMINATOR`.

Recommendations

Cap the `userBonusBps` so that the total split cannot exceed 100%. This will maintain the `totalSplit <= BPS_DENOMINATOR` invariant for all `setUserBonusSplit` calls, but will not maintain the invariant across all users for `setProfitSplit` calls. To maintain this invariant in all cases, and allow users to always withdraw profits, we can cap the `totalSplit` at 100% in `processPayout`:

```
uint256 totalSplit = profitSplit + _bonusBps;  
if (totalSplit > BPS_DENOMINATOR) totalSplit = BPS_DENOMINATOR;
```